

1.1 Schulverwaltungssystem

Erstellen Sie ein einfaches Schulverwaltungssystem mit folgenden Klassen:

- Schueler
- Lehrer
- Kurs
- Schule

Anforderungen:

- Die Klasse " Schueler " soll Attribute wie Name, Alter und eine Liste von belegten Kursen haben.
- Die Klasse " Lehrer " soll Attribute wie Name, Fachbereich und eine Liste von unterrichteten Kursen haben.
- Die Klasse " Kurs " soll Attribute wie Name, Raumnummer und eine Referenz zum unterrichtenden Lehrer haben.
- Die Klasse " Schule " soll Listen von Schuelern, Lehrern und Kursen haben sowie Methoden zum Hinzufuegen und Entfernen von Schuelern und Lehrern.

Implementieren Sie geeignete Methoden in jeder Klasse, wie z.B. das Hinzufuegen eines Schuelers zu einem Kurs oder das Zuweisen eines Kurses zu einem Lehrer. Stellen Sie die Beziehungen zwischen den Klassen her.

Erstellung eines UML-Klassendiagramms fuer ein Schulverwaltungssystem

1. ueerblick und Zielsetzung

Ein UML-Klassendiagramm dient dazu, die Struktur eines Systems zu modellieren. Es stellt Klassen, ihre Attribute und Methoden sowie die Beziehungen zwischen ihnen dar. In dieser Aufgabe soll ein Diagramm fuer ein Schulverwaltungssystem erstellt werden, das die Klassen *Schueler*, *Lehrer*, *Kurs* und *Schule* umfasst.

2. Schritte zur Erstellung

Schritt 1: Identifikation der Klassen

Die Klassen ergeben sich direkt aus den Anforderungen:

- **Schueler**
- **Lehrer**
- **Kurs**
- **Schule**

Jede dieser Klassen erfuehlt spezifische Aufgaben und hat eigene Attribute und Methoden.

Schritt 2: Definition der Attribute und Methoden

Analysieren Sie die Anforderungen, um die Attribute und Methoden fuer jede Klasse zu bestimmen.

2.1 Schueler

- **Attribute:**
 - **name:** Der Name des Schuelers (**String**).
 - **alter:** Das Alter des Schuelers (**int**).
 - **kurse:** Eine Liste der Kurse, die der Schueler belegt hat (**List<Kurs>**).
- **Methode:**
 - **kursBelegen(kurs: Kurs):** Fuegt einen Kurs zur Liste der belegten Kurse hinzu.

2.2 Lehrer

- **Attribute:**

- **name:** Der Name des Lehrers (**String**).
- **fachbereich:** Der Fachbereich des Lehrers (**String**).
- **kurse:** Eine Liste der Kurse, die der Lehrer unterrichtet (**List<Kurs>**).

- **Methode:**

- **kursHinzufuegen(kurs: Kurs):** Fuegt einen Kurs zur Liste der unterrichteten Kurse hinzu.

2.3 Kurs

- **Attribute:**

- **name:** Der Name des Kurses (**String**).
- **raumnummer:** Die Raumnummer, in der der Kurs stattfindet (**String**).
- **lehrer:** Eine Referenz auf den Lehrer, der den Kurs unterrichtet (**Lehrer**).
- **schueler:** Eine Liste der Schueler, die den Kurs besuchen (**List<Schueler>**).

- **Methode:**

- **schuelerHinzufuegen(schueler: Schueler):** Fuegt einen Schueler zur Liste der Teilnehmer hinzu.

2.4 Schule

- **Attribute:**

- **schueler:** Eine Liste aller Schueler in der Schule (**List<Schueler>**).
- **lehrer:** Eine Liste aller Lehrer in der Schule (**List<Lehrer>**).
- **kurse:** Eine Liste aller angebotenen Kurse (**List<Kurs>**).

- **Methoden:**

- **schuelerHinzufuegen(schueler: Schueler):** Fuegt einen Schueler zur Schule hinzu.
- **lehrerHinzufuegen(lehrer: Lehrer):** Fuegt einen Lehrer zur Schule hinzu.

Schritt 3: Darstellung der Beziehungen

Identifizieren Sie die Beziehungen zwischen den Klassen:

Aggregation:

- **Schule** besteht aus *Schuelern*, *Lehrern* und *Kursen*. Diese Beziehung ist eine Aggregation, da die Schule ohne die einzelnen Komponenten existieren kann.

Assoziation:

- Ein **Kurs** hat genau einen *Lehrer* (1:1).
- Ein **Kurs** hat viele *Schueler* (1:n).

Schritt 4: Erstellung des Diagramms

Nutzen Sie eine UML-Software oder eine LaTeX-Umgebung oder Mermaid, um das Diagramm zu zeichnen. Das Diagramm wird in Rechtecken unterteilt:

- Der oberste Bereich enthaelt den Klassennamen.
- Der mittlere Bereich listet die Attribute auf.
- Der unterste Bereich zeigt die Methoden.

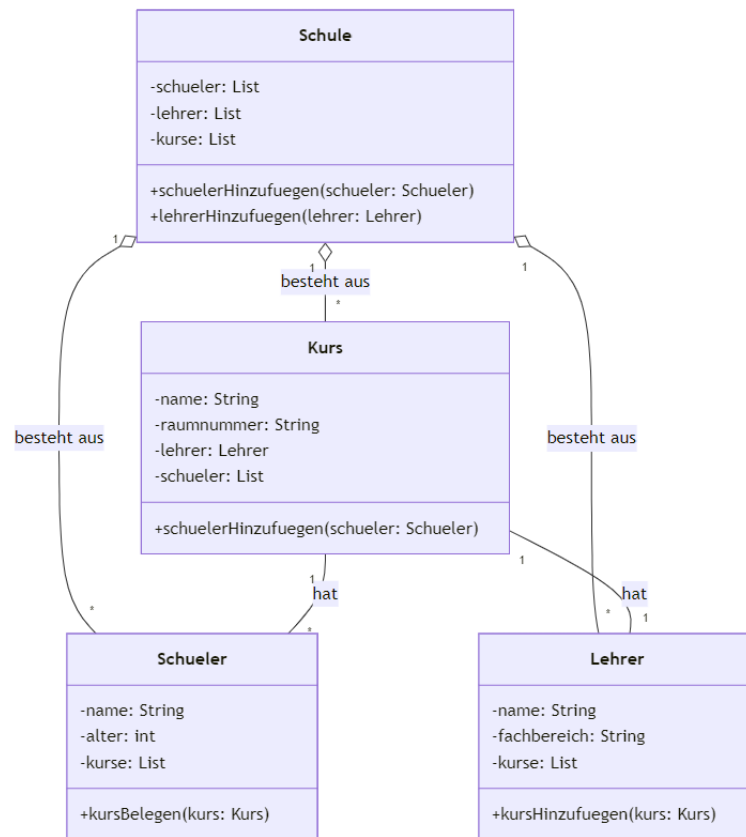


Abbildung 1: UML-Klassendiagramm des Schulverwaltungssystems

```

1 class Schueler:
2     def __init__(self, name, alter):
3         self.name = name
4         self.alter = alter
5         self.kurse = []
6
7     def kurs_belegen(self, kurs):
8         if kurs not in self.kurse:
9             self.kurse.append(kurs)
10            kurs.schueler_hinzufuegen(self)
11
12    def kurs_verlassen(self, kurs):
13        if kurs in self.kurse:
14            self.kurse.remove(kurs)
15            kurs.schueler_entfernen(self)
16
17 class Lehrer:
18     def __init__(self, name, fachbereich):
19         self.name = name
20         self.fachbereich = fachbereich
21         self.kurse = []
22
23     def kurs_hinzufuegen(self, kurs):
24         if kurs not in self.kurse:
25             self.kurse.append(kurs)
26             kurs.lehrer_zuweisen(self)
27
28     def kurs_entfernen(self, kurs):
29         if kurs in self.kurse:
30             self.kurse.remove(kurs)
31             kurs.lehrer_entfernen()
32
33 class Kurs:
34     def __init__(self, name, raumnummer):
35         self.name = name
36         self.raumnummer = raumnummer
37         self.lehrer = None
38         self.schueler = []
39
40     def lehrer_zuweisen(self, lehrer):
41         self.lehrer = lehrer
42
43     def lehrer_entfernen(self):
44         self.lehrer = None
45
46     def schueler_hinzufuegen(self, schueler):
47         if schueler not in self.schueler:
48             self.schueler.append(schueler)
49
50     def schueler_entfernen(self, schueler):
51         if schueler in self.schueler:
52             self.schueler.remove(schueler)
53
54 class Schule:
55     def __init__(self, name):
56         self.name = name
57         self.schueler = []

```

```

58     self.lehrer = []
59     self.kurse = []
60
61     def schueler_hinzufuegen(self, schueler):
62         if schueler not in self.schueler:
63             self.schueler.append(schueler)
64
65     def schueler_entfernen(self, schueler):
66         if schueler in self.schueler:
67             self.schueler.remove(schueler)
68             for kurs in schueler.kurse:
69                 kurs.schueler_entfernen(schueler)
70
71     def lehrer_hinzufuegen(self, lehrer):
72         if lehrer not in self.lehrer:
73             self.lehrer.append(lehrer)
74
75     def lehrer_entfernen(self, lehrer):
76         if lehrer in self.lehrer:
77             self.lehrer.remove(lehrer)
78             for kurs in lehrer.kurse:
79                 kurs.lehrer_entfernen()
80
81     def kurs_erstellen(self, name, raumnummer):
82         kurs = Kurs(name, raumnummer)
83         self.kurse.append(kurs)
84         return kurs
85
86     def kurs_entfernen(self, kurs):
87         if kurs in self.kurse:
88             self.kurse.remove(kurs)
89             for schueler in kurs.schueler:
90                 schueler.kurs_verlassen(kurs)
91             if kurs.lehrer:
92                 kurs.lehrer.kurs_entfernen(kurs)
93
94 schule = Schule("Mustergymnasium")
95
96 lehrer1 = Lehrer("Herr Mueller", "Mathematik")
97 schule.lehrer_hinzufuegen(lehrer1)
98
99 schueler1 = Schueler("Anna Schmidt", 16)
100 schule.schueler_hinzufuegen(schueler1)
101
102 kurs1 = schule.kurs_erstellen("Mathematik 10", "Raum 101")
103 lehrer1.kurs_hinzufuegen(kurs1)
104 schueler1.kurs_belegen(kurs1)
105
106 print(f"Schule: {schule.name}")
107 print(f"Lehrer: {lehrer1.name}, Fachbereich: {lehrer1.fachbereich}"
108       )
109 print(f"Schueler: {schueler1.name}, Alter: {schueler1.alter}")
110 print(f"Kurs: {kurs1.name}, Raum: {kurs1.raumnummer}")
111 print(f"Lehrer des Kurses: {kurs1.lehrer.name}")
112 print(f"Schueler im Kurs: {[s.name for s in kurs1.schueler]}")

```

1 Erweitertes Schulverwaltungssystem

Eine mögliche Erweiterung könnte die nachfolgende Gestalt haben.

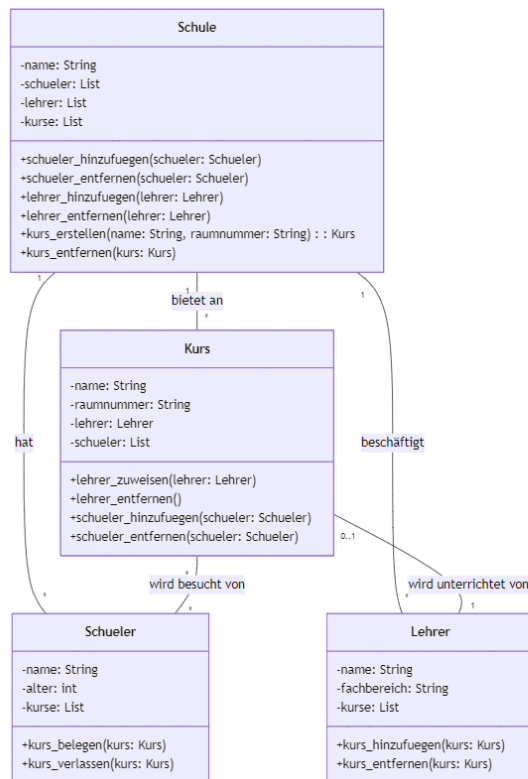


Abbildung 2: UML-Klassendiagramm des erweiterten Schulverwaltungssystems